

Software Architecture and Design

From Analysis to Architecture
Monolithic → Clean → Microservices

Tran Dinh Que

Software Architecture and Design

Assignment 1 — Due in 2 Weeks

Course Roadmap

- ① From Analysis to Design and Architecture
- ② Architectural Styles and Evolution
- ③ Case Study: E-commerce Bookstore

Reference: https://www.tutorialspoint.com/software_architecture_design/introduction.htm

What Is Requirement Analysis?

- Requirement Collection?
- Requirement Determination?
- Requirement Analysis?
- Which UML diagrams should be used for Analysis?

Problems in Requirement analysis

- NOT know use case, sequence + activity diagrams for what?
- NOT know HOW to build suitable class diagram and use for what
- NOT understand the differences of class diagram in analysis and in design

What Is Software Architecture?

Architecture Definition

Software architecture is the set of fundamental structures of a system, their relationships, and guiding principles.

- High-level organization
- Long-term design decisions
- Quality attributes focus

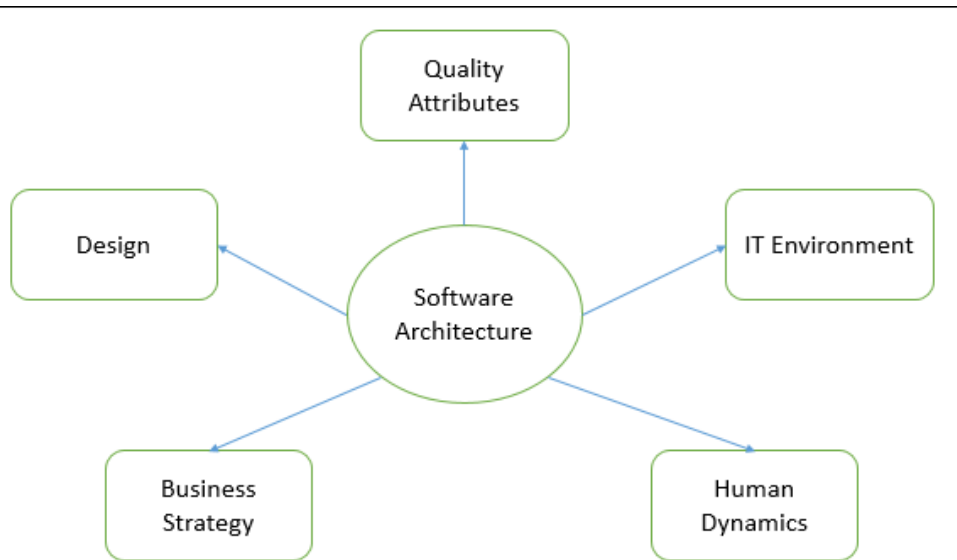
Analysis vs Design vs Architecture

- **Analysis:** What problem are we solving?
- **Design:** How do components interact?
- **Architecture:** What structure governs the system?

Key Insight

Architecture constrains and guides detailed design.

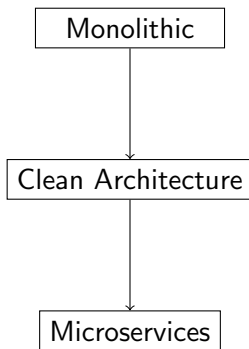
Software Architecture Overview



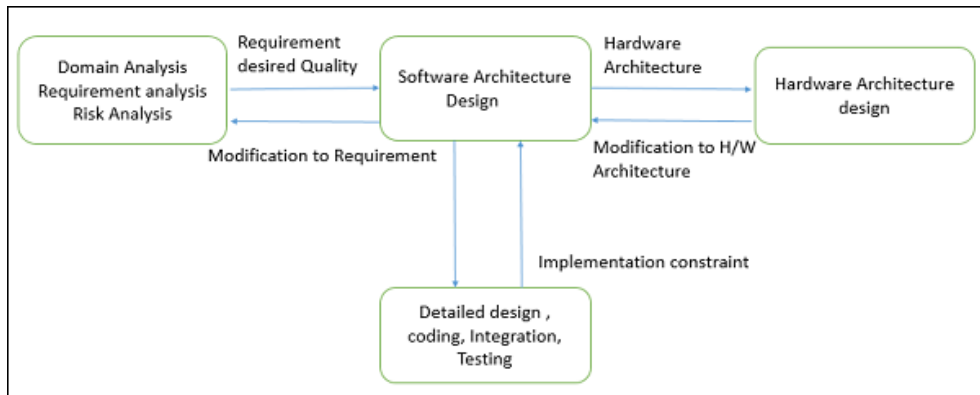
Architectural Styles

- Layered Architecture
- Client–Server
- Event-Driven
- Clean Architecture
- Microservices

Architecture Evolution



Software Architecture and Design Overview



Source: Software Architecture and Design (Conceptual Overview)

Monolithic Architecture

- Single deployable unit
- Shared database
- Tight coupling

<https://www.geeksforgeeks.org/system-design/monolithic-architecture-system-design/>

<https://www.techtarget.com/whatis/definition/monolithic-architecture>

Advantages

Simple to build, simple to deploy initially

Clean Architecture

- Business logic at the center
- Frameworks are external
- Dependency inversion

<https://www.freecodecamp.org/news/a-quick-introduction-to-clean-architecture-990c014448d2/>

Microservices

<https://microservices.io/patterns/microservices.html>

- Independent services
- Independent databases
- Independent deployment

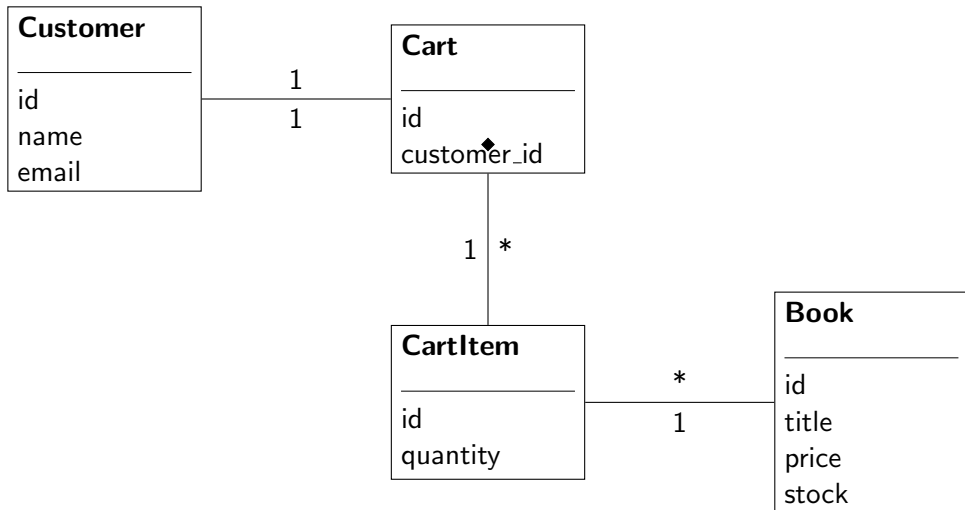
Cost

Operational and communication complexity

- Customer
- Cart
- CartItem
- Book

Use case: **Add Book to Cart**

UML Class Diagram (Reference Model)



Why CartItem Is Necessary

- Supports quantity per book
- Supports pricing snapshots
- Supports discounts and promotions

Design Principle

Avoid embedding collections directly inside entities.

Monolithic Django Example

```
def add_to_cart(request, customer_id, book_id):  
    book = Book.objects.get(id=book_id)  
    cart = Cart.objects.get(customer_id=customer_id)  
    CartItem.objects.create(  
        cart=cart,  
        book=book,  
        quantity=1  
    )
```


Clean Architecture — Use Case

```
class AddBookToCartUseCase:
    def execute(self, customer_id, book_id):
        book = self.book_repo.get(book_id)
        cart = self.cart_repo.get_by_customer(customer_id)
        cart.add_book(book)
        self.cart_repo.save(cart)
```

Microservice Decomposition

- Customer Service
- Book Service
- Cart Service

Communication via REST or events.

Architecture Comparison

Aspect	Monolith	Clean	Microservices
Complexity	Low	Medium	High
Scalability	Low	Medium	High
Testability	Low	High	High
Deployment	Single	Single	Multiple

Assignment 1 — Student Tasks

- ① Redraw UML diagrams using Visual Paradigm
- ② Implement Add-to-Cart in Monolithic Django
- ③ Refactor to Clean Architecture
- ④ Propose Microservice decomposition

Questions?